

Teórica 7 — Resumo para Frequência

Modelos Probabilísticos · Ensemble Models

PARTE 1 · Naive Bayes

1.1 Teorema de Bayes

Descreve como atualizar a probabilidade de uma hipótese com base em nova evidência, combinando conhecimento prévio (prior) com os dados observados (likelihood):

$$P(B|A) = P(A|B) \cdot P(B) / P(A)$$

Termo	Nome	Significado
P(B)	Prior	Probabilidade inicial antes de ver evidência
P(A B)	Likelihood	Prob. de observar A se B for verdade
P(B A)	Posterior	Crença atualizada depois de ver A
P(A)	Evidência	Normalizador (garante que soma = 1)

Problema: Com muitas features é preciso aplicar a chain rule a todas as combinações — computacionalmente inviável e exige dados que normalmente não existem.

1.2 Naive Bayes — a solução

Naive Independence Assumption: assume-se que todas as features X são estatisticamente independentes (mesmo que não o sejam na realidade). Isto colapsa a chain rule para:

$$P(C_k | x_1, \dots, x_n) \propto P(C_k) \cdot \prod P(x_i | C_k)$$

A classificação passa a ser uma simples multiplicação de likelihoods individuais pelo prior da classe. Escolhe-se a classe que maximiza este produto.

Exemplo: Para prever burnout alto, em vez de calcular a probabilidade conjunta de todas as features simultaneamente, calcula-se separadamente $P(\text{study_hours}=x | \text{burnout_alto})$, $P(\text{sleep_hours}=y | \text{burnout_alto})$, etc., e multiplica tudo.

1.3 Categorical Naive Bayes

Usado quando as features são **categóricas** (valores discretos).

$$P(C | A, B) = P(A|C) \cdot P(B|C) \cdot P(C) / [P(A|C) \cdot P(B|C) \cdot P(C) + P(A|\neg C) \cdot P(B|\neg C) \cdot P(\neg C)]$$

Exemplo (COVID, dois testes A e B positivos): $P(\text{Covid+})=0.357 \cdot P(A+|\text{Covid+})=0.8 \cdot P(B+|\text{Covid+})=0.8 \rightarrow P(\text{Covid+}|A+,B+) = 0.878$

1.4 Problema: Zero Probabilities → Laplace Smoothing

Se uma feature nunca foi observada para uma classe no treino, a sua likelihood é 0 e **anula todo o produto** — uma única feature nunca vista elimina uma classe inteira.

Solução — Add- α Smoothing:

$$P(X=x | C) = (\text{count}(x, C) + \alpha) / (\text{count}(C) + \alpha \cdot K)$$

Parâmetro	Descrição
α	Parâmetro de smoothing
K	Número de valores possíveis da feature
$\alpha=1$	Laplace Smoothing — assume cada valor observado pelo menos uma vez

- Previne zeros e reduz overfitting
- α grande → probabilidades mais uniformes (mais regularização)

1.5 Gaussian Naive Bayes (features contínuas)

Para features contínuas, valores exatos têm probabilidade zero — não faz sentido calcular $P(\text{temperatura} = 38.4^\circ\text{C exatamente})$. Modela-se cada feature como uma **Gaussiana** por classe, aprendendo μ e σ^2 no treino:

$$p(x_i | C_k) = (1 / \sqrt{2\pi \sigma^2}) \cdot \exp(-(x_i - \mu)^2 / (2\sigma^2))$$

Na predição calcula-se a densidade gaussiana (não probabilidade). Exemplo: paciente com febre= 38.4°C ($\mu_{\text{Covid+}}=38.5$) → densidade alta ≈ 0.78 vs febre ($\mu_{\text{Covid-}}=36.5$) → densidade ≈ 0.0001 → classifica Covid+.

1.6 Propriedades do Naive Bayes

Propriedade	Resposta	Razão
Simples?	✓ Sim	Apenas contagens e ratios; $M \times K$ parâmetros
Estável?	✓ Sim	Resultado sempre igual, independente da ordem
Rápido a treinar?	✓ Sim	Uma passagem pelos dados (contagens básicas)
Rápido a prever?	✓ Sim	Produto de likelihoods e prior
Atualizável?	✓ Sim	Adicionar dados = incrementar contagens

PARTE 2 · Ensemble Models

Intuição Base

Ideia fundamental: um conjunto de modelos simples pode superar um único modelo sofisticado. Modelos simples são rápidos, paralelizáveis, e a sua repetição extrai insights sobre os dados que por vezes são mais valiosos do que o modelo em si.

2.1 Bootstrapping

Bootstrap = criar um dataset do mesmo tamanho do original por amostragem **com reposição** (o mesmo exemplo pode aparecer várias vezes).

Out-of-Bag (OOB) = exemplos do dataset original que *não* foram incluídos no bootstrap → servem como conjunto de validação natural, sem necessidade de separar um validation set adicional.

2.2 Bagging — Bootstrap Aggregating

Treina K modelos independentes em datasets bootstrapped diferentes e agrega as previsões.

1. Criar dataset bootstrap B (com reposição)
2. Criar dataset OOB (os que ficaram de fora)
3. Treinar modelo fraco em B
4. Avaliar no OOB
5. Repetir K vezes
6. Majority voting (classificação) ou média (regressão)

***Porquê funciona?** Reduz a variância sem aumentar o bias. Uma árvore única é muito sensível ao ruído; a média de várias árvores treinadas em subsets ligeiramente diferentes não o é — desde que as árvores não sejam correlacionadas.*

2.3 Random Forests

O algoritmo de bagging mais popular. Usa Decision Trees como weak models. Adicionalmente ao bootstrap, seleciona-se também um **subset aleatório de features** em cada split — garante que as árvores não são correlacionadas.

- Bootstrap dos dados
- Selecionar aleatoriamente um subset das colunas (features)
- Treinar uma DT (tipicamente sem pruning)
- Repetir K vezes → combinar por majority voting / média

***Hiperparâmetros:** todos os da DT + K (número de árvores).*

2.4 Pasting

Como bagging mas a amostragem é **sem reposição** — cada subset contém exemplos únicos. Decisão final: modo (classificação) ou média (regressão).

2.5 Boosting — Ideia Geral

Completamente diferente do bagging: modelos treinados **sequencialmente**, cada novo modelo foca-se em corrigir os erros do anterior. Não é constrangido algorítmicamente — qualquer weak model pode ser usado.

1. Treinar modelo fraco → 2. Identificar erros → 3. Dar mais peso às instâncias mal classificadas → 4. Treinar novo modelo focado nesses casos → 5. Repetir → 6. Previsão final = soma ponderada dos weak learners

2.6 AdaBoost (Adaptive Boosting)

Mecanismo de **pesos nas instâncias**:

- Inicialmente todos os exemplos têm peso = $1/N$
- Após cada modelo: instâncias mal classificadas recebem peso maior
- O peso do classificador na decisão final (alpha) é proporcional ao seu acerto
- **Não é paralelizável** — sequencial por natureza

Analogia: como um professor que dá mais atenção aos alunos que erram repetidamente.

2.7 Gradient Boosting

Em vez de ajustar pesos nas instâncias, cada modelo é treinado para **prever os resíduos (erros)** do modelo anterior.

$$Result = R1 + (1-r) \cdot R2 + (1-r)^2 \cdot R3 + \dots + (1-r)^{(k-1)} \cdot Rk$$

O **learning rate (r)** escala a contribuição de cada modelo subsequente — modelos mais tardios contribuem menos, o que evita overfitting.

Analogia: afinar uma escultura — o 1.º modelo faz a forma grosseira, o 2.º corrige as maiores imperfeições, o 3.º as mais pequenas, etc.

2.8 XGBoost — eXtreme Gradient Boosting

Implementação otimizada do Gradient Boosting com melhorias chave:

Melhoria	O que faz
Regularização	Penaliza complexidade → menos overfitting
2ª Ordem (Hessiana)	Usa gradiente E curvatura → updates mais precisos e estáveis
Shrinkage	Cada árvore contribui pouco → melhor generalização
Missing Values	Aprende automaticamente para onde enviar NaN nos splits
Paralelização	Split finding paralelizado dentro de cada árvore

2.9 Stacking (Stacked Generalization)

Combina **modelos diferentes** (não o mesmo modelo repetido) e aprende a combiná-los com um **meta-modelo**.

- Treinar múltiplos base models independentemente (ex: DT + KNN + SVM)
- As previsões de cada base model formam um novo dataset (meta-dataset)
- Treinar um meta-model (level-1) nesse meta-dataset
- Para novos dados: base models preveem → meta-model combina

Diferença key vs bagging/boosting: stacking usa modelos diferentes e aprende o peso de cada um em vez de usar majority vote fixo ou soma ponderada por erro.

2.10 Comparação Final

Algoritmo	Paralel.	Sequencial	Atualizável	Nota
Random Forests	✓	✗	✗	Mais popular; robusto
AdaBoost	✗	✓	✗	Requer weak models simples
Gradient Boost	✗	✓	✗	Mais flexível que AdaBoost
XGBoost	Parcial	✓	✗	Melhor desempenho geral atualmente
Stacking	✓	✗	✗	Combina modelos diferentes

Pontos-Chave para a Frequência

- **Naive Bayes** = Bayes Theorem + independence assumption → produto de likelihoods

- **Categorical NB** para features discretas; **Gaussian NB** para contínuas (usa densidade gaussiana)
- **Laplace smoothing** ($\alpha=1$) para evitar probabilidades zero
- **Bagging** (paralelo, com reposição) vs **Boosting** (sequencial, corrige erros)
- **Random Forest** = Bagging + DTs + subset aleatório de features
- **AdaBoost** = pesos nas instâncias; **Gradient Boosting** = prevê resíduos
- **XGBoost** = Gradient Boosting + regularização + Hessiana + paralelização
- **Stacking** = meta-modelo aprende a combinar modelos diferentes
- Nenhum ensemble é actualizável — novo dado pode mudar todos os modelos internos